LinkGuard: Phishing Link Warning System

Class group: CS3 - (CS GROUP 4)
Team: Nicholas Groenewald, Nikoloz Chilachava, Ibrahima Toure Ba, Solomon Sosanya

What problem are we solving?
Most scams now start with a link. A text says your parcel is held, your bank account is locked or your Revenue refund is waiting and the link takes you to a fake site built to steal your login or card details. These sites are getting harder to spot because they copy real logos, use HTTPS, and pick web addresses that look almost identical to the real thing.
The problem we want to solve: how can an ordinary person tell whether a link is safe before they click it and hand over their details?

Why is it important?
Phishing hits everyone, but it hurts most for the people who can't spot it, like the elderly or anyone who isn't confident online. A single click can lead to drained bank accounts, stolen identities, or compromised accounts. Current advice, “check the URL carefully”, asks users to act like security experts. A tool that checks a link for them and explains the warning signs in plain language would protect people and also teach them what to look out for next time.

What is the ultimate goal?
Our goal is a web application where a user pastes a suspicious link and gets back:

- a risk level (safe, suspicious or high risk)
- the specific reasons the link looks dangerous, such as an unusual domain, misleading page title, or hidden login forms
- clear advice on what to do next

To build this, we will use the PhiUSIIL Phishing URL Dataset from the UCI Machine Learning Repository. It contains 235,795 labelled URLs (134,850 legitimate and 100,945 phishing) with 54 features taken from both the URL and the webpage's source code and was published in 2024, so it reflects current phishing tactics. We will train and compare classification models on this data, then use the best one to power the app.

How we will deliver it
The system will have a React frontend, a Python FastAPI backend, and a trained machine learning model. We will build it incrementally across Agile sprints:

1. Explore and clean the dataset, set up a basic website
2. Train a first model and classify pasted links
3. Add a risk score to the result
4. Explain which features triggered the warning
5. Improve the interface and add scan history and user feedback

Future work could include a URL DNS details and other features to help with improved detection.